

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2026/2027

MODUL 10- State, Hooks (useState, useEffect), dan Conditional Rendering

Nama Mahasiswa : Aisyah Ayu Salsabila
NIM : 23051430001
Kelas : A2_Teknik Industri
Tanggal Praktikum : 25 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10 (Pertemuan 10)
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	24 April 2026
Nama Mahasiswa	Aisyah Ayu Salsabila
NIM	23051430001
Kelas	A2_Teknik Industri

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi Live Server		Perubahan web realtime
4	Laptop	Windows 11	Prasarana Praktikum
5	Modul Aplikasi Web dan Mobile	2025/2026	Manual Praktikum
6	Console browser dan ekstensi Quokka.js		JavaScript alat pemroses data

7.	Internet		Mengakses API publik
8.	Node.js		Untuk npm
9.	Project React		Hasil Pertemuan 10

D. DASAR TEORI

1. Latar Belakang Penggunaan State dalam Konteks Teknik Industri

Dalam sistem industri modern, data tidak pernah bersifat statis. Sebuah dashboard produksi, misalnya, harus mampu menampilkan jumlah unit yang telah diproduksi, tingkat efisiensi mesin (OEE), atau status inventori secara *real-time*. Jika antarmuka hanya menampilkan data statis (read-only), pengguna tidak dapat melihat perubahan yang terjadi secara langsung—mereka harus me-refresh halaman secara manual, yang tidak efisien dan tidak sesuai dengan kebutuhan pengambilan keputusan cepat di rantai produksi.

Pada aplikasi berbasis web tradisional (Vanilla JavaScript), perubahan data memerlukan manipulasi DOM secara manual dan seringkali menyebabkan kode menjadi rumit, sulit dirawat, dan rawan kesalahan, terutama ketika sistem memiliki banyak elemen yang saling bergantung. Misalnya, sebuah dashboard dengan 10 tampilan berbeda yang semuanya harus diperbarui ketika satu data sensor berubah.

Untuk mengatasi hal tersebut, React menyediakan mekanisme **state**, yaitu data internal yang dapat berubah (*mutable*) dan ketika berubah akan secara otomatis me-render ulang (*re-render*) seluruh komponen yang bergantung padanya. Dengan state, aplikasi industri dapat menjadi **responsif, dinamis**, dan **real-time** tanpa kehilangan keteraturan kode.

2. State vs Props dalam Sistem Industri

Dalam React, terdapat dua mekanisme utama untuk mengelola data: **Props** dan **State**.

- **Props (Properties):** Data yang dikirim dari komponen induk ke komponen anak. Bersifat *read-only* (tidak dapat diubah oleh penerima). Analoginya seperti **instruksi kerja standar (SOP)** yang diberikan dari manajemen ke operator. Operator harus mengikuti instruksi tersebut tanpa bisa mengubahnya.
- **State:** Data yang dimiliki dan dikelola sepenuhnya oleh komponen itu sendiri. Bersifat *read-write* (dapat diubah oleh komponen pemilikinya). Analoginya seperti **papan informasi produksi** di stasiun kerja. Operator dapat menulis ulang jumlah produksi yang telah dicapai setiap saat, dan setiap perubahan langsung terlihat oleh semua orang.

3. useState Hook untuk Membuat State dan useEffect Hook untuk Side Effects

useState adalah Hook dasar untuk menambahkan state ke komponen fungsional.

Sintaks:

```
const [nilai, setNilai] = useState(initialValue);
```

- nilai → nilai state saat ini.

- `setNilai` → fungsi untuk mengubah nilai state.
- `initialValue` → nilai awal.

`useEffect` digunakan untuk menangani efek samping, seperti:

- Mengambil data dari API sensor secara periodik.
- Menjalankan timer untuk simulasi waktu siklus.
- Membersihkan koneksi saat komponen dihapus.

Berbeda dengan props, state adalah data yang dimiliki dan dikelola oleh komponen itu sendiri.

Karakteristik:

- Bersifat read-write (dapat diubah).
- Ketika state berubah, React otomatis re-render tampilan.

Analogi

Teknik

Industri:

State seperti papan informasi produksi di stasiun kerja. Operator dapat memperbarui jumlah produksi yang telah dicapai, dan setiap perubahan langsung terlihat oleh semua orang.

4. Props sebagai Mekanisme Distribusi Data

Props (Properties) digunakan untuk mengirim data dari komponen induk ke komponen anak.

Karakteristik:

- Bersifat read-only (tidak dapat diubah oleh penerima).
- Digunakan untuk membuat komponen menjadi dinamis namun tetap terkendali.

Props seperti instruksi kerja standar (SOP) dari manajemen ke operator. Operator menerima dan mengikuti instruksi tersebut tanpa bisa mengubahnya.

5. Conditional Rendering untuk Tampilan Adaptif

Conditional rendering adalah teknik menampilkan elemen berbeda berdasarkan kondisi (biasanya nilai state).

Metode umum:

Metode	Sintaks	Penggunaan
Ternary	{kondisi ? <A /> : }	Pilihan dua tampilan
AND (&&)	{kondisi && <Component />}	Tampil hanya jika true
if	if (kondisi) return <A />	Blok besar

Seperti lampu indikator pada panel mesin:

- Hijau jika suhu normal.
- Merah jika suhu melebihi batas.
- Berkedip jika sensor bermasalah.

6. Siklus Hidup (Lifecycle) Komponen

Komponen React memiliki tiga fase siklus hidup yang diimplementasikan dengan `useEffect`:

Fase	Analogi Industri	Implementasi
Mount	Mesin mulai beroperasi	<code>useEffect(() => {...}, [])</code>
Update	Mesin menyesuaikan parameter	<code>useEffect(() => {...}, [dependency])</code>
Unmount	Mesin dimatikan & dibersihkan	<code>return () => {...}</code>

D.1 Konsep Utama

State management dalam React terdiri dari tiga pilar:

1. `useState` Memberikan "memori" pada komponen untuk data yang berubah.
2. `useEffect` Menangani efek samping (timer, API, koneksi sensor).
3. Conditional Rendering Menampilkan tampilan adaptif berdasarkan kondisi.

D.2 Keterkaitan dengan Teknik Industri

Penguasaan state management memungkinkan mahasiswa Teknik Industri membangun dashboard produksi real-time, simulasi antrian, kalkulator biaya dinamis, dan monitoring IoT sederhana dengan memanfaatkan `useState`, `useEffect`, serta conditional rendering. Ketiga konsep ini menjadi fondasi untuk menciptakan antarmuka yang responsif terhadap perubahan data di rantai produksi. Dengan demikian, mahasiswa mampu mengembangkan alat bantu digital interaktif untuk monitoring, simulasi, dan optimasi sistem nyata secara efisien.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Buat file baru `src/Komponen/CounterProduksi.jsx`, kemudian meng import CounterProduksi di App

```
import React, { useState } from 'react';

function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  const [emergency, setEmergency] = useState(false);

  const tambahProduksi = () => {
    if (!emergency) {
      setJumlah(jumlah + 1);
    }
  };

  const reset = () => {
    setJumlah(0);
  };
}
```

```

    setEmergency(false);
  };

  const emergencyStop = () => {
    setEmergency(true);
  };

  return (
    <div className="text-center p-4 border rounded bg-light shadow-sm">
      <h3> Simulasi Hitung Produk</h3>
      <h1 className="display-1 fw-bold text-primary">{jumlah}</h1>
      <p className="text-muted">Target: {target} Unit</p>

      {emergency ? (
        <div className="alert alert-danger d-inline-block fw-bold">
          EMERGENCY STOP! PRODUKSI DIHENTIKAN
        </div>
      ) : jumlah >= target ? (
        <div className="alert alert-success d-inline-block">
          Target Tercapai! Selamat!
        </div>
      ) : (
        <div className="alert alert-secondary d-inline-block">
          Produksi Berjalan...
        </div>
      )}

      <div className="mt-3">
        <button className="btn btn-primary me-2" onClick={tambahProduksi}
disabled={emergency}>
          +1 Unit
        </button>
        <button className="btn btn-warning me-2" onClick={reset}>
          Reset Shift
        </button>
        <button className="btn btn-danger" onClick={emergencyStop}
disabled={emergency}>
          Emergency Stop
        </button>
      </div>

      <div className="mt-3">
        <div className="progress" style={{ height: '30px' }}>
          <div
            className={`progress-bar ${jumlah >= target ? 'bg-success' :
'bg-primary'} fw-bold`}

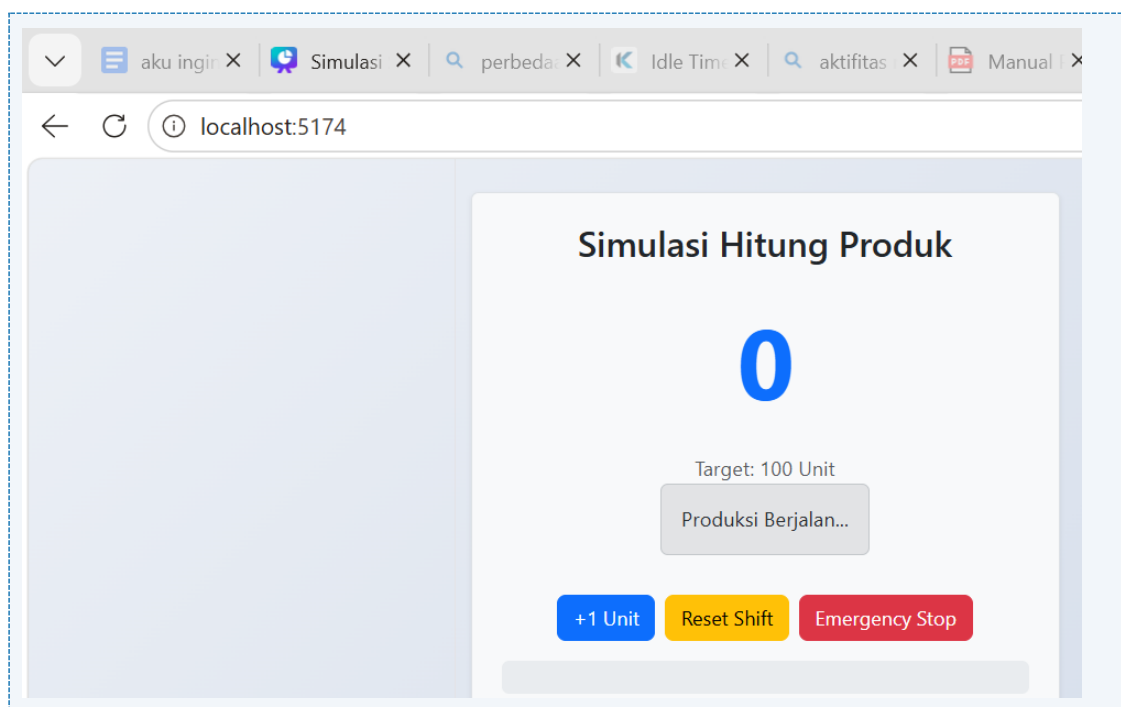
```

```

        style={{ width: `${Math.min((jumlah / target) * 100, 100)}%` }}
      >
        {Math.min(Math.round((jumlah / target) * 100), 100)}%
      </div>
    </div>
  </div>
</div>
);
}

export default CounterProduksi;

```



Gambar E.1 – Counter Produksi

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

UseEffect untuk Real-time dengan membuat jam digital yang berjalan otomatis. Ini mensimulasikan pengambilan data live dari sensor tiap detik. t komponen baru 'JamDigital.jsx'

```

import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  const [kota, setKota] = useState('Yogyakarta');

  useEffect(() => {

```

```

const timerID = setInterval(() => {
  setWaktu(new Date());
}, 1000);
return () => clearInterval(timerID);
}, []);

useEffect(() => {
  document.title = `Jam ${kota} - ${waktu.toLocaleTimeString()}`;
}, [kota, waktu]);

return (
  <div className="alert alert-info text-center shadow-sm">
    <h4> Waktu Server Real-time</h4>
    <h1 className="display-4 fw-bold">{waktu.toLocaleTimeString()}</h1>
    <p>{waktu.toLocaleDateString('id-ID', { weekday: 'long', year:
'numeric', month: 'long', day: 'numeric' })}</p>
    <div className="mt-2">
      <label className="fw-bold me-2"> Lokasi Pabrik:</label>
      <input
        type="text"
        className="form-control d-inline-block w-auto"
        value={kota}
        onChange={(e) => setKota(e.target.value)}
        placeholder="Masukkan nama kota"
      />
      <small className="d-block text-muted mt-1">Judul tab akan berubah
sesuai kota</small>
    </div>
  </div>
);
}

export default JamDigital;

```

Screenshot Hasil:



Gambar E.2 – Jam Didital

E.3 Langkah 3 – Contoh Form Input State

Buat file baru di folder `src` bernama `Komponen/KartuMesin.jsx`.

```
import React, { useState } from 'react';

function KartuMesin({ nama, status: statusAwal, produksi: produksiAwal }) {
  const [statusLokal, setStatusLokal] = useState(statusAwal);
  const [produksi, setProduksi] = useState(produksiAwal);

  const getBadgeColor = () => {
    switch (statusLokal) {
      case 'Running': return 'bg-success';
      case 'Stop': return 'bg-danger';
      case 'Maintenance': return 'bg-warning text-dark';
      default: return 'bg-secondary';
    }
  };

  const tambahProduksi = () => {
    if (statusLokal === 'Running') {
      setProduksi(produksi + 1);
    }
  };

  return (
    <div className="card shadow-sm h-100">
      <div className="card-body">
        <div className="d-flex justify-content-between align-items-start">
          <h5 className="card-title">{nama}</h5>
          <span className={`badge ${getBadgeColor()}`}>{statusLokal}</span>
        </div>
        <hr />
        <div className="text-center my-3">
          <h2 className="fw-bold text-primary">{produksi}</h2>
          <small className="text-muted">Unit</small>
        </div>
        <select
          className="form-select form-select-sm mb-2"
          value={statusLokal}
          onChange={(e) => setStatusLokal(e.target.value)}
        >
          <option value="Running"> Running</option>
          <option value="Stop"> Stop</option>
          <option value="Maintenance"> Maintenance</option>
        </select>
        <button
```

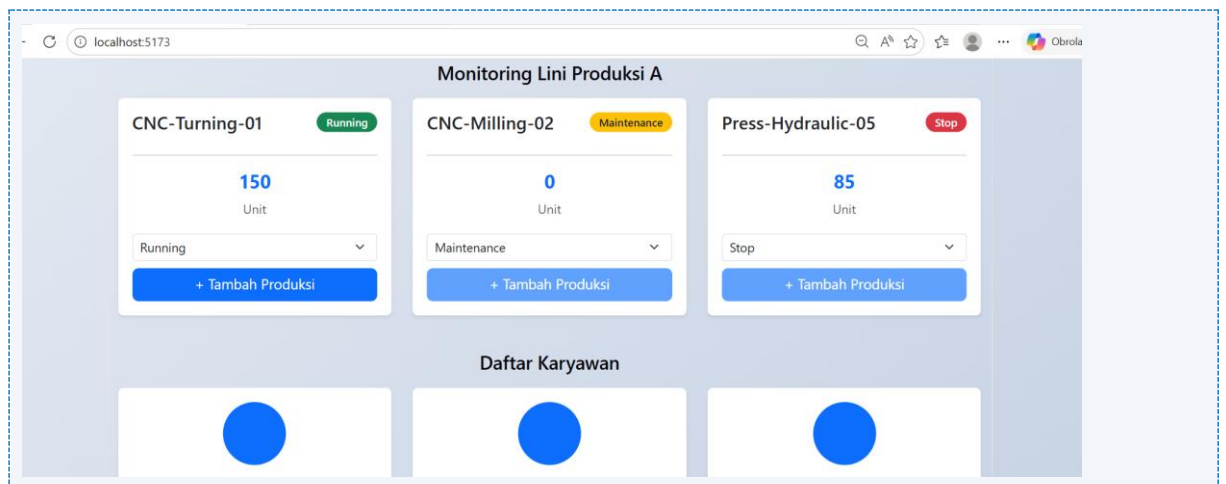
```

        className="btn btn-primary w-100"
        onClick={tambahProduksi}
        disabled={statusLokal !== 'Running'}
      >
        + Tambah Produksi
      </button>
    </div>
  </div>
);
}

export default KartuMesin;

```

Screenshot Hasil:



Gambar E.3 – tampilan kartu mesin

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Latihan 1 (Dependency Array useEffect): Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`

Jawaban / Kode Solusi:

```

import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  const [kota, setKota] = useState('Yogyakarta');

```

```
useEffect(() => {
  const timerID = setInterval(() => {
    setWaktu(new Date());
  }, 1000);
  return () => clearInterval(timerID);
}, []);

useEffect(() => {
  document.title = `Jam ${kota} - ${waktu.toLocaleTimeString()}`;
}, [kota, waktu]);
```



Gambar F.1 – Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Latihan 2 (Conditional Rendering): Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React from 'react';
import React, { useState } from 'react';

function KartuMesin({ nama, status: statusAwal, produksi: produksiAwal }) {
  {
    const [statusLokal, setStatusLokal] = useState(statusAwal);
    const [produksi, setProduksi] = useState(produksiAwal);
```

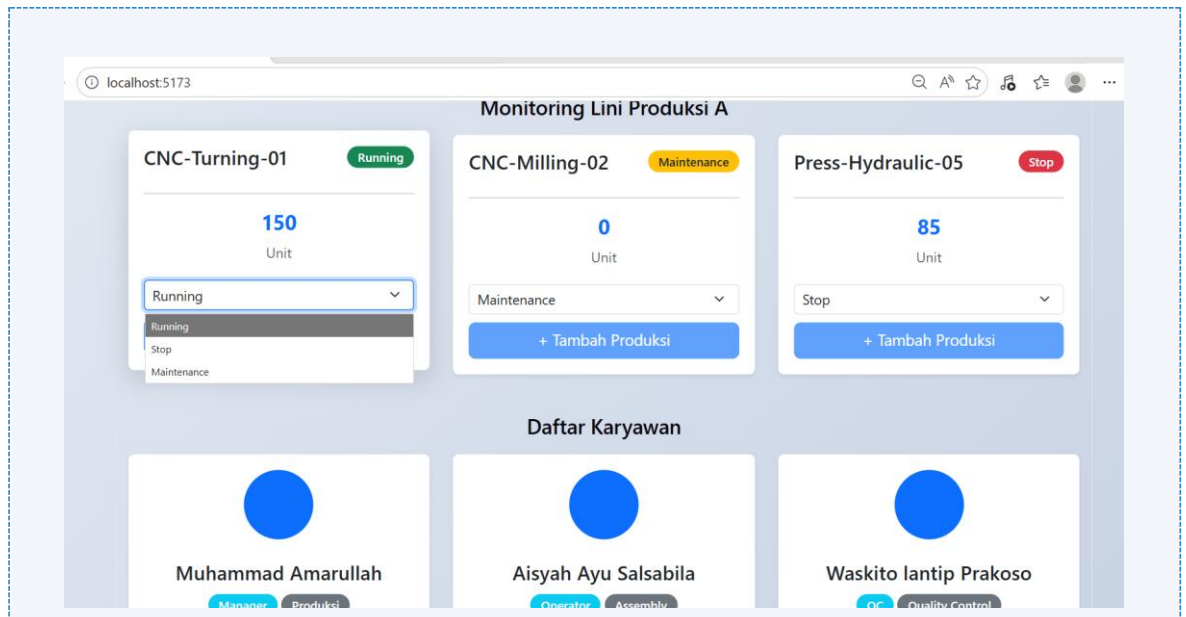
```
const getBadgeColor = () => {
  switch (statusLokal) {
    case 'Running': return 'bg-success';
    case 'Stop': return 'bg-danger';
    case 'Maintenance': return 'bg-warning text-dark';
    default: return 'bg-secondary';
  }
};

const tambahProduksi = () => {
  if (statusLokal === 'Running') {
    setProduksi(produksi + 1);
  }
};

return (
  <div className="card shadow-sm h-100">
    <div className="card-body">
      <div className="d-flex justify-content-between align-items-start">
        <h5 className="card-title">{nama}</h5>
        <span className={`badge
${getBadgeColor()}`}>{statusLokal}</span>
      </div>
      <hr />
      <div className="text-center my-3">
        <h2 className="fw-bold text-primary">{produksi}</h2>
        <small className="text-muted">Unit</small>
      </div>
      <select
        className="form-select form-select-sm mb-2"
        value={statusLokal}
        onChange={(e) => setStatusLokal(e.target.value)}
      >
        <option value="Running"> Running</option>
        <option value="Stop"> Stop</option>
        <option value="Maintenance"> Maintenance</option>
      </select>
      <button
        className="btn btn-primary w-100"
        onClick={tambahProduksi}
        disabled={statusLokal !== 'Running'}
      >
        + Tambah Produksi
      </button>
    </div>
  </div>
```

```
);
}

export default KartuMesin;
```



Gambar F.2 – Conditional Rendering

F.3 Tugas Proyek Mini – (Kartu Karyawan)

Deskripsi proyek mini:

Tugas Proyek Mini (Kalkulator OEE Sederhana): a. Buat komponen form input: 'Plan Time', 'Run Time', 'Total Parts', 'Good Parts'. b. Gunakan State untuk menyimpan nilai-nilai ini. c. Hitung OEE secara otomatis (Real-time) setiap kali input berubah. d. Rumus OEE = (Availability x Performance x Quality). e. Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Membuat komponen baru yaitu kalkulator oee, state sebagai penyimpanan nilai, OEE terhitung otomatis dengan input, menghasilkan persenn dan tanda warna.

Jawaban untuk app.jsx:

```
import React, { useState, useEffect } from 'react';

function KalkulatorOEE() {
  const [planTime, setPlanTime] = useState(480);
  const [runTime, setRunTime] = useState(420);
  const [totalParts, setTotalParts] = useState(500);
  const [goodParts, setGoodParts] = useState(480);
```

```

const [oee, setOee] = useState(0);
const [availability, setAvailability] = useState(0);
const [performance, setPerformance] = useState(0);
const [quality, setQuality] = useState(0);

useEffect(() => {
  const avail = (runTime / planTime) * 100;
  const perf = ((totalParts / runTime) / 1) * 100;
  const qual = (goodParts / totalParts) * 100;
  const oeeValue = (avail / 100) * (perf / 100) * (qual / 100) * 100;

  setAvailability(avail);
  setPerformance(perf);
  setQuality(qual);
  setOee(oeeValue);
}, [planTime, runTime, totalParts, goodParts]);

const getOeeColor = () => {
  if (oee >= 85) return 'success';
  if (oee >= 65) return 'primary';
  if (oee >= 50) return 'warning';
  return 'danger';
};

return (
  <div className="card shadow-sm">
    <div className="card-header bg-dark text-white">
      <h4 className="mb-0"> Kalkulator OEE (Overall Equipment
Effectiveness)</h4>
    </div>
    <div className="card-body">
      <div className="row">
        <div className="col-md-6">
          <label className="form-label fw-bold">Plan Time
(menit)</label>
          <input type="number" className="form-control"
value={planTime} onChange={(e) => setPlanTime(Number(e.target.value))}
/>

          <label className="form-label fw-bold mt-2">Run Time
(menit)</label>
          <input type="number" className="form-control"
value={runTime} onChange={(e) => setRunTime(Number(e.target.value))} />

          <label className="form-label fw-bold mt-2">Total Parts
(unit)</label>

```

```

        <input type="number" className="form-control"
value={totalParts} onChange={(e) =>
setTotalParts(Number(e.target.value))} />

        <label className="form-label fw-bold mt-2">Good Parts
(unit)</label>
        <input type="number" className="form-control"
value={goodParts} onChange={(e) =>
setGoodParts(Number(e.target.value))} />
      </div>
      <div className="col-md-6">
        <div className="mb-3">
          <p>Availability:
<strong>{availability.toFixed(2)}%</strong></p>
          <div className="progress"><div className="progress-bar
bg-info" style={{ width: `${availability}%` }}></div></div>
          </div>
          <div className="mb-3">
            <p>Performance:
<strong>{performance.toFixed(2)}%</strong></p>
            <div className="progress"><div className="progress-bar
bg-primary" style={{ width: `${Math.min(performance, 100)}%`
}}></div></div>
            </div>
            <div className="mb-3">
              <p>Quality: <strong>{quality.toFixed(2)}%</strong></p>
              <div className="progress"><div className="progress-bar
bg-success" style={{ width: `${quality}%` }}></div></div>
              </div>
              <div className={`alert alert-${getOeeColor()} text-center
mt-3`} >
                <h2>OEE: {oee.toFixed(2)}%</h2>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}

export default KalkulatorOEE;

```

Kode Untuk APP.jsx

```

import React from 'react';
import CounterProduksi from './Komponen/CounterProduksi';
import JamDigital from './Komponen/JamDigital';
import KartuMesin from './Komponen/KartuMesin';           // ← Perbaiki
ini!
import KalkulatorOEE from './Komponen/KalkulatorOEE';
import KartuKaryawan from './Komponen/KartuKaryawan';    // ← Tambahkan
ini juga

function App() {
  return (
    <div className="container-fluid py-4">
      <div className="text-center mb-4">
        <h1 className="display-5 fw-bold text-primary">
Monitoring Lini Produksi A</h1>
        <p className="lead">Aisyah Ayu Salsabila -23051430001
</p>
        <hr />
      </div>

      <div className="row mb-4">
        <div className="col-12">
          <JamDigital />
        </div>
      </div>

      <div className="row mb-4">
        <div className="col-md-6">
          <CounterProduksi />
        </div>
        <div className="col-md-6">
          <KalkulatorOEE />
        </div>
      </div>

      <div className="row">
        <div className="col-md-4 mb-3">
          <KartuMesin
            nama="CNC-Turning-01"
            status="Running"

```



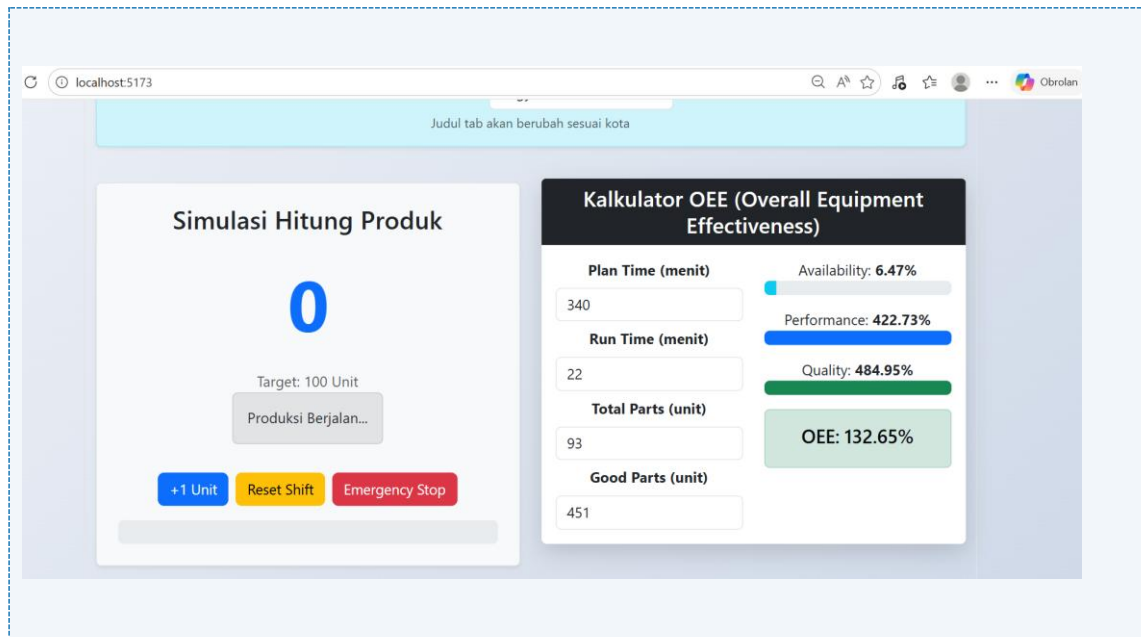
```

        produksi={150}
      />
    </div>
    <div className="col-md-4 mb-3">
      <KartuMesin
        nama="CNC-Milling-02"
        status="Maintenance"
        produksi={0}
      />
    </div>
    <div className="col-md-4 mb-3">
      <KartuMesin
        nama="Press-Hydraulic-05"
        status="Stop"
        produksi={85}
      />
    </div>
  </div>

  { /* Kartu Karyawan */ }
  <h2 className="text-center mb-3 mt-4"> Daftar Karyawan</h2>
  <div className="row">
    <div className="col-md-4 mb-3">
      <KartuKaryawan
        nama="Muhammad Amarullah"
        jabatan="Manager"
        bagian="Produksi"
      />
    </div>
    <div className="col-md-4 mb-3">
      <KartuKaryawan
        nama="Aisyah Ayu Salsabila"
        jabatan="Operator"
        bagian="Assembly"
      />
    </div>
    <div className="col-md-4 mb-3">
      <KartuKaryawan
        nama="Waskito lantip Prakoso"
        jabatan="QC"
        bagian="Quality Control"
      />
    </div>
  </div>
</div>
);

```

```
}  
  
export default App;
```



Gambar F.3 – Daftar Karyawan

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum telah sesuai dengan teori React Hooks, di mana `useState` berhasil mengelola state dinamis seperti jumlah produksi dan status darurat, `useEffect` mampu menjalankan efek samping berupa timer real-time dan update title tab secara otomatis, serta conditional rendering menampilkan pesan berbeda berdasarkan kondisi state.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kendala utama adalah error "does not provide an export named 'default'" karena semua file komponen di folder Komponen tidak memiliki pernyataan export default di akhir file, sehingga React tidak dapat mengimpor komponen tersebut.	Menambahkan export default NamaKomponen; pada setiap file komponen (CounterProduksi, JamDigital, KartuMesin, KalkulatorOEE, KartuKaryawan).
folder Komponen sempat berada di luar folder src yang menyebabkan path import salah	sehingga harus dipindahkan ke dalam folder src..

Tracking Prevention memblokir akses storage untuk Bootstrap CDN.	Install Bootstrap lokal dengan npm install bootstrap lalu import di main.jsx.
Komponen tidak muncul karena folder Komponen berada di luar folder src.	Pindahkan folder Komponen ke dalam folder src agar path import ./Komponen/... dapat terbaca.
State tidak berubah meskipun tombol ditekan karena lupa menggunakan functional update.	Gunakan setJumlah(prev => prev + 1) bukan setJumlah(jumlah + 1) untuk menghindari masalah asynchronous.

G.3 Kaitan dengan Penerapan di Industri

Praktikum ini relevan dengan industri karena penggunaan state memungkinkan monitoring data produksi secara real-time, seperti jumlah unit yang dihasilkan mesin CNC. `useEffect` dapat diterapkan untuk mengambil data sensor secara periodik atau update dashboard otomatis tanpa refresh halaman. Conditional rendering berguna untuk menampilkan peringatan darurat (emergency stop) atau status maintenance mesin yang langsung terlihat oleh operator. Secara keseluruhan, komponen-komponen seperti Kalkulator OEE dan Kartu Mesin mencerminkan sistem informasi produksi yang digunakan di pabrik modern untuk meningkatkan efisiensi dan pengambilan keputusan.

H. KESIMPULAN

1	State adalah memori komponen React yang membedakannya dengan Props (read-only), di mana State bersifat read-write dan setiap perubahannya akan memicu re-render otomatis pada tampilan.
2	Hook <code>useState</code> digunakan untuk mendeklarasikan state variable dengan sintaks <code>const [nilai, setNilai] = useState(0)</code> , dan untuk update yang bergantung pada nilai lama wajib menggunakan functional update <code>setCount(prev => prev + 1)</code> .
3	Hook <code>useEffect</code> berfungsi menangani side effects seperti timer atau API call, dengan dependency array kosong <code>[]</code> berarti efek hanya berjalan sekali saat komponen dimount, serta wajib memiliki cleanup function untuk mencegah memory leak.
4	Conditional rendering memungkinkan tampilan berubah berdasarkan kondisi state, seperti menampilkan alert berbeda ketika target produksi tercapai, serta tombol "+1" menjadi disabled saat mode emergency stop diaktifkan.
5	Penerapan state, effect, dan conditional rendering sangat relevan dengan industri, misalnya untuk simulasi hitung produksi, jam digital real-time sebagai pengganti sensor, serta kalkulator OEE yang menghitung efektivitas mesin secara otomatis berdasarkan input plan time, run time, total parts, dan good parts.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
-----	------------------------------------

[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chambers, J. M. (2014). <i>Object-Oriented Programming , Functional Programming and R</i> . 29(2), 167–180. https://doi.org/10.1214/13-STS452
[3]	Gunawan, R., Suherman, Y., & Wibowo, S. S. (2022). <i>Rancang Bangun Sistem Informasi Verifikasi Dan Validasi Data Pengajuan Tender Berbasis Web Menggunakan Model Terstruktur dan UML Menurut Instruksi Presiden Nomor 3 Tahun mengembangkan</i> . 16(62), 10–19.
[4]	Mulawarman Munsyir, S.E., S.Si., M. K., Eko Prasetyo Widhi, M. K., Eko Wahyudi, M. T., Dr. Bagus Setya Rintyarna, M. K., & Vandika, A. Y. (2024). <i>Belajar Algoritma dan Pemrograman dengan Javascript</i> .
[5]	Kusnawi, Romadhon, W., & Dhuhiha, W. M. P. (2024). <i>Menyelami dunia React.JS: Best practice berdasarkan pengalaman di industri IT</i> . Penerbit ANDI.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa



Aisyah Ayu Salsabila
NIM: 23051430001

Diperiksa oleh Asisten / Dosen

Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.
Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —